

Problem 3: Penguin Species Discovery with K-Means

This notebook implements k-means from scratch in NumPy and applies it to the Palmer Penguins morphology features in `penguins.csv`. Species labels are held out during clustering and used only afterward for interpretation.

(a) Exploratory Data Analysis

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.metrics import silhouette_score

sns.set_theme(style="whitegrid", context="notebook")
plt.rcParams["figure.figsize"] = (8, 5)

DATA_PATH = "penguins.csv"
FEATURES = ["bill_length_mm", "bill_depth_mm", "flipper_length_mm", "body_mass_g"]

penguins = pd.read_csv(DATA_PATH, na_values=["NA"])

print("Shape:", penguins.shape)
display(penguins.describe(include="all"))

print("Penguins per species:")
display(penguins["species"].value_counts().rename("count").to_frame())

print("Penguins per island:")
display(penguins["island"].value_counts().rename("count").to_frame())

print("Missing values per column:")
missing_by_column = penguins.isna().sum().rename("missing_count").to_frame()
display(missing_by_column)

clean_penguins = penguins.dropna(subset=FEATURES).copy()
print(f"Rows remaining after dropping NaN in morphology features: {len(clean_penguins)}")

plt.figure(figsize=(8, 5.5))
sns.scatterplot(
    data=clean_penguins,
    x="flipper_length_mm",
    y="bill_length_mm",
    hue="species",
    style="species",
    s=70,
    edgecolor="white",
```

```
linewidth=0.5,
)
plt.title("Penguins by Recorded Species")
plt.xlabel("Flipper length (mm)")
plt.ylabel("Bill length (mm)")
plt.tight_layout()
plt.show()
```

Shape: (344, 8)

	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	s
count	344	344	342.000000	342.000000	342.000000	342.000000	3
unique	3	3	NaN	NaN	NaN	NaN	
top	Adelie	Biscoe	NaN	NaN	NaN	NaN	m
freq	152	168	NaN	NaN	NaN	NaN	1
mean	NaN	NaN	43.921930	17.151170	200.915205	4201.754386	N
std	NaN	NaN	5.459584	1.974793	14.061714	801.954536	N
min	NaN	NaN	32.100000	13.100000	172.000000	2700.000000	N
25%	NaN	NaN	39.225000	15.600000	190.000000	3550.000000	N
50%	NaN	NaN	44.450000	17.300000	197.000000	4050.000000	N
75%	NaN	NaN	48.500000	18.700000	213.000000	4750.000000	N
max	NaN	NaN	59.600000	21.500000	231.000000	6300.000000	N



Penguins per species:

count	
species	
Adelie	152
Gentoo	124
Chinstrap	68

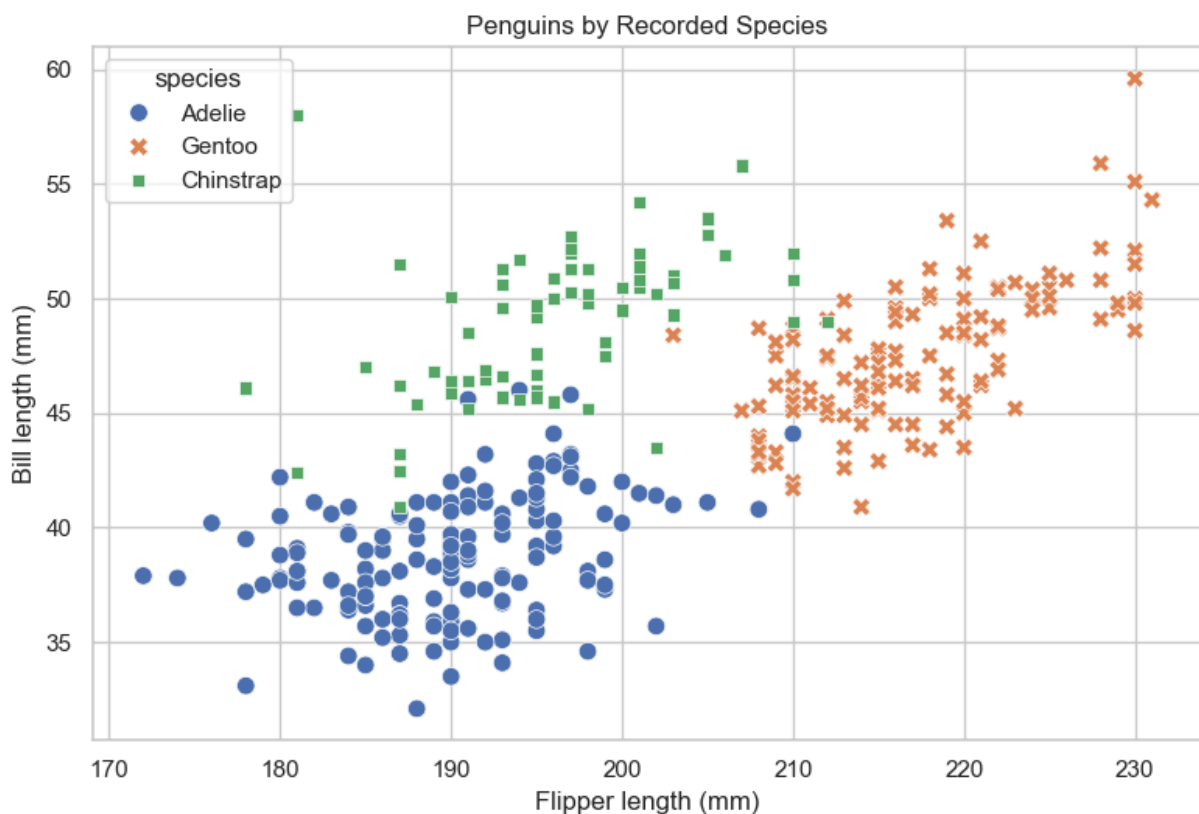
Penguins per island:

count	
island	
Biscoe	168
Dream	124
Torgersen	52

Missing values per column:

missing_count	
species	0
island	0
bill_length_mm	2
bill_depth_mm	2
flipper_length_mm	2
body_mass_g	2
sex	11
year	0

Rows remaining after dropping NaN in morphology features: 342



There is a clear Gentoo blob with longer flippers, while Adelie and Chinstrap overlap more in this two-dimensional projection. Chinstrap tends to have a longer bill than Adelie, but those two species are the most similar by eye in the flipper-length vs. bill-length plot.

(b) Preprocessing

```
In [2]: X_original_df = clean_penguins[FEATURES].astype(float).copy()
true_species = clean_penguins["species"].copy()

feature_means = X_original_df.mean(axis=0)
```

```

feature_stds = X_original_df.std(axis=0, ddof=0)
X = ((X_original_df - feature_means) / feature_stds).to_numpy()

scaled_check = pd.DataFrame(
    {
        "scaled_mean": X.mean(axis=0),
        "scaled_std": X.std(axis=0, ddof=0),
    },
    index=FEATURES,
)
display(scaled_check)

print("Original feature means used for standardization:")
display(feature_means.rename("mean").to_frame())
print("Original feature standard deviations used for standardization:")
display(feature_stds.rename("std").to_frame())

```

	scaled_mean	scaled_std
bill_length_mm	1.662088e-16	1.0
bill_depth_mm	4.155221e-16	1.0
flipper_length_mm	-8.310441e-16	1.0
body_mass_g	8.310441e-17	1.0

Original feature means used for standardization:

	mean
bill_length_mm	43.921930
bill_depth_mm	17.151170
flipper_length_mm	200.915205
body_mass_g	4201.754386

Original feature standard deviations used for standardization:

	std
bill_length_mm	5.451596
bill_depth_mm	1.971904
flipper_length_mm	14.041141
body_mass_g	800.781229

The four clustering features are `bill_length_mm`, `bill_depth_mm`, `flipper_length_mm`, and `body_mass_g`; categorical columns and the held-out `species` label are excluded. Standardization is necessary because k-means uses Euclidean distance: without scaling, `body_mass_g` would dominate simply because it is measured in thousands, while bill measurements are much smaller numeric values.

(c) Implement K-Means from Scratch

```
In [3]: def _kmeans_internal(X, k, max_iter=100, seed=0, return_history=False):
    X = np.asarray(X, dtype=float)
    n_samples = X.shape[0]
    if k < 1 or k > n_samples:
        raise ValueError("k must be between 1 and the number of samples")

    rng = np.random.default_rng(seed)
    initial_idx = rng.choice(n_samples, size=k, replace=False)
    centroids = X[initial_idx].copy()
    labels = np.full(n_samples, -1, dtype=int)
    inertia_history = []

    for _ in range(max_iter):
        distances_sq = ((X[:, None, :] - centroids[None, :, :]) ** 2).sum(axis=2)
        new_labels = np.argmin(distances_sq, axis=1)

        new_centroids = centroids.copy()
        for cluster_id in range(k):
            members = X[new_labels == cluster_id]
            if len(members) > 0:
                new_centroids[cluster_id] = members.mean(axis=0)
            else:
                new_centroids[cluster_id] = X[rng.integers(n_samples)]

        updated_distances_sq = ((X - new_centroids[new_labels]) ** 2).sum(axis=1)
        inertia_history.append(float(updated_distances_sq.sum()))

        if np.array_equal(new_labels, labels):
            centroids = new_centroids
            labels = new_labels
            break

        centroids = new_centroids
        labels = new_labels

    final_distances_sq = ((X[:, None, :] - centroids[None, :, :]) ** 2).sum(axis=2)
    final_labels = np.argmin(final_distances_sq, axis=1)
    final_inertia = float(final_distances_sq[np.arange(n_samples), final_labels].sum())

    if not inertia_history or not np.isclose(inertia_history[-1], final_inertia):
        inertia_history.append(final_inertia)

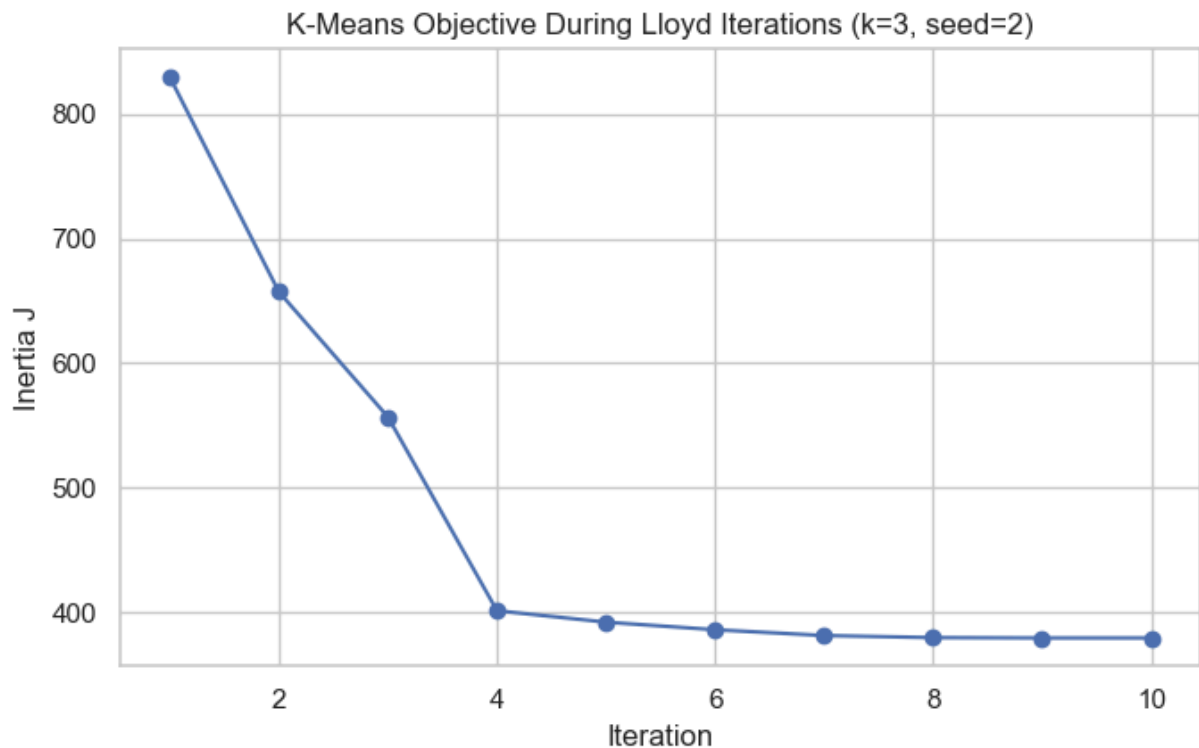
    if return_history:
        return centroids, final_labels, final_inertia, inertia_history
    return centroids, final_labels, final_inertia

def kmeans(X, k, max_iter=100, seed=0):
    # Required assignment signature.
    return _kmeans_internal(X, k, max_iter=max_iter, seed=seed, return_history=False)
```

```
centroids_demo, labels_demo, inertia_demo, history_demo = _kmeans_internal(
    X, k=3, max_iter=100, seed=2, return_history=True
)

plt.figure(figsize=(7, 4.5))
plt.plot(range(1, len(history_demo) + 1), history_demo, marker="o")
plt.title("K-Means Objective During Lloyd Iterations (k=3, seed=2)")
plt.xlabel("Iteration")
plt.ylabel("Inertia J")
plt.tight_layout()
plt.show()

print("Inertia history:", np.round(history_demo, 4))
print("Monotone non-increasing:", bool(np.all(np.diff(history_demo) <= 1e-10)))
```



```
Inertia history: [829.5432 657.2344 556.8323 401.2819 392.0808 386.0062 381.349 379.6756
 379.3925 379.3925]
Monotone non-increasing: True
```

The plotted objective is monotone non-increasing, as expected from Lloyd's assignment and centroid-update steps.

(d) Cluster and Validate

```
In [4]: main_seed = 2
centroids_k3, labels_k3, inertia_k3 = kmeans(X, k=3, seed=main_seed)
print(f"Final inertia for k=3 with seed={main_seed}: {inertia_k3:.3f}")

cluster_species_table = pd.crosstab(
    pd.Series(labels_k3, name="cluster"),
```

```

    true_species.rename("species"),
)
display(cluster_species_table)

purity = cluster_species_table.max(axis=1).sum() / cluster_species_table.to_numpy()
print(f"Clustering purity: {purity:.3f}")

centroids_original = centroids_k3 * feature_stds.to_numpy() + feature_means.to_numpy()
centroid_table = pd.DataFrame(centroids_original, columns=FEATURES)
centroid_table.index.name = "cluster"
display(centroid_table.round(2))

```

Final inertia for k=3 with seed=2: 379.393

species Adelie Chinstrap Gentoo

cluster

0	24	61	2
1	126	5	0
2	1	0	121

Clustering purity: 0.906

bill_length_mm bill_depth_mm flipper_length_mm body_mass_g

cluster

0	47.53	18.76	196.90	3902.01
1	38.21	18.11	188.40	3584.66
2	47.50	14.98	217.19	5076.02

Cluster 0 is a Chinstrap-like group: long bills, deep bills, moderate flippers, and medium body mass. Cluster 1 is an Adelie-like group: shorter bills, deep bills, shorter flippers, and smaller body mass. Cluster 2 is a Gentoo-like group: large body mass, long flippers, and comparatively shallow bills.

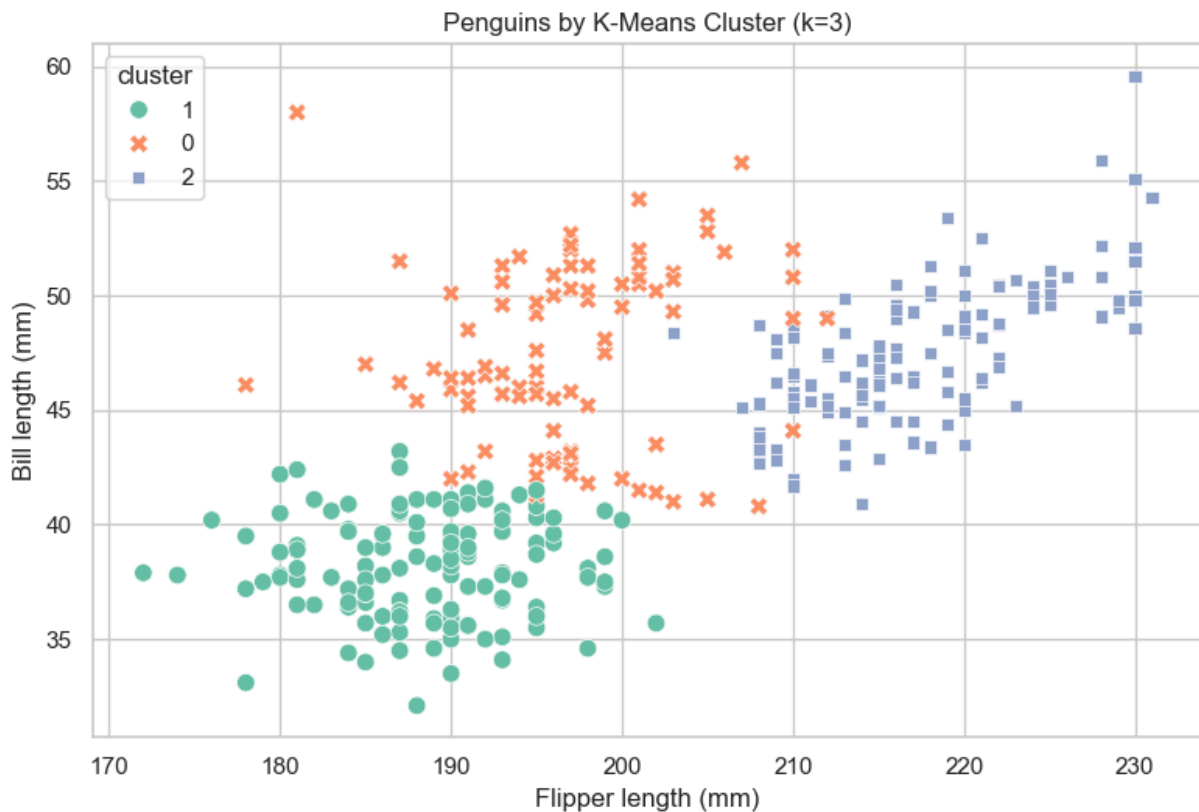
In [5]: `plot_df = clean_penguins.assign(cluster=labels_k3.astype(str))`

```

plt.figure(figsize=(8, 5.5))
sns.scatterplot(
    data=plot_df,
    x="flipper_length_mm",
    y="bill_length_mm",
    hue="cluster",
    style="cluster",
    palette="Set2",
    s=70,
    edgecolor="white",
    linewidth=0.5,
)
plt.title("Penguins by K-Means Cluster (k=3)")
plt.xlabel("Flipper length (mm)")

```

```
plt.ylabel("Bill length (mm)")
plt.tight_layout()
plt.show()
```



The cluster-colored scatter mostly resembles the species-colored version: the Gentoo-like group is cleanly separated, while Adelie and Chinstrap overlap in the center-left of the plot. The main differences occur along the Adelie/Chinstrap boundary, where morphology is less cleanly separated in these two dimensions.

(e) The Elbow Method

```
In [6]: elbow_rows = []
for k in range(1, 11):
    inertias = [kmeans(X, k=k, seed=seed)[2] for seed in range(10)]
    elbow_rows.append(
        {
            "k": k,
            "mean_inertia": float(np.mean(inertias)),
            "std_inertia": float(np.std(inertias)),
            "best_inertia": float(np.min(inertias)),
        }
    )

elbow_df = pd.DataFrame(elbow_rows)
display(elbow_df.round(3))

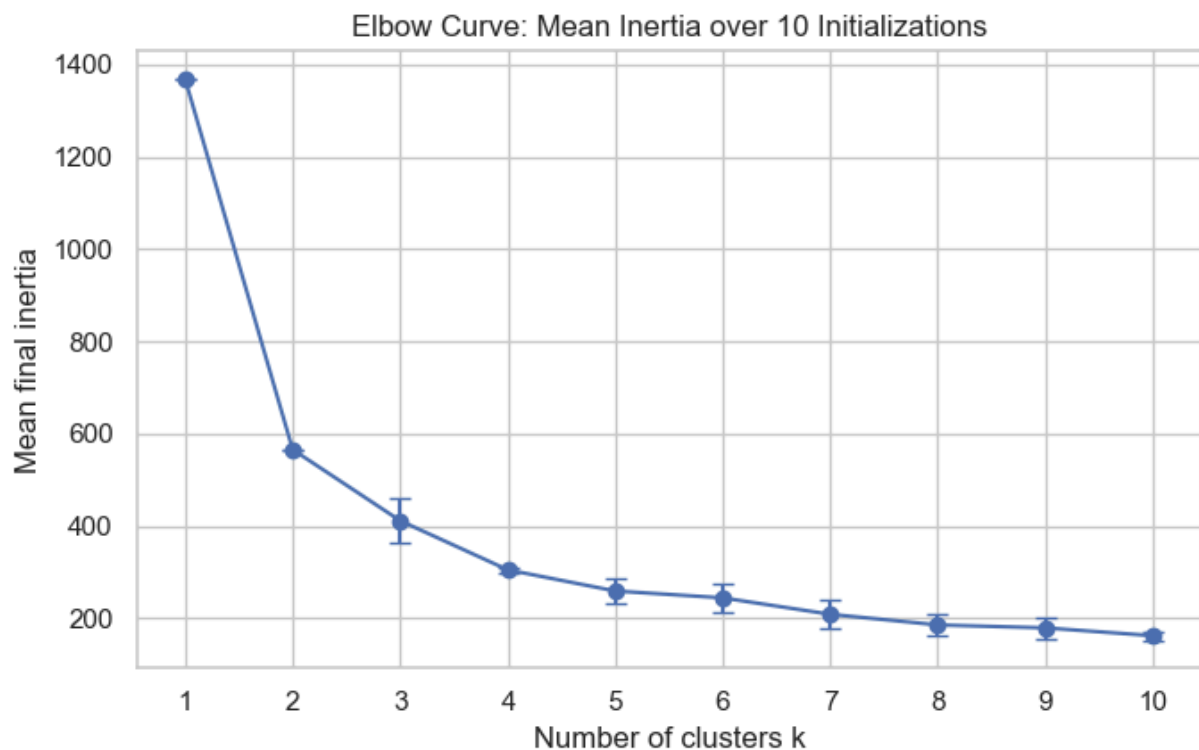
plt.figure(figsize=(7, 4.5))
plt.errorbar(
```

```

    elbow_df["k"],
    elbow_df["mean_inertia"],
    yerr=elbow_df["std_inertia"],
    marker="o",
    capsize=4,
)
plt.xticks(range(1, 11))
plt.title("Elbow Curve: Mean Inertia over 10 Initializations")
plt.xlabel("Number of clusters k")
plt.ylabel("Mean final inertia")
plt.tight_layout()
plt.show()

```

	k	mean_inertia	std_inertia	best_inertia
0	1	1368.000	0.000	1368.000
1	2	565.708	0.000	565.708
2	3	411.627	48.865	379.393
3	4	305.269	5.222	300.400
4	5	259.483	27.127	232.597
5	6	244.537	30.338	205.127
6	7	209.003	30.523	187.155
7	8	185.960	22.712	171.368
8	9	179.548	23.879	160.739
9	10	162.613	9.241	152.667



The largest improvement is from `k=1` to `k=2`, and a smaller but still meaningful bend appears around `k=3`. This is partly consistent with the three recorded species, but it also reveals that the strongest morphology split is Gentoo versus non-Gentoo, while Adelie and Chinstrap are harder to separate.

(f) Silhouette Score

```
In [7]: silhouette_rows = []
best_runs_by_k = {}

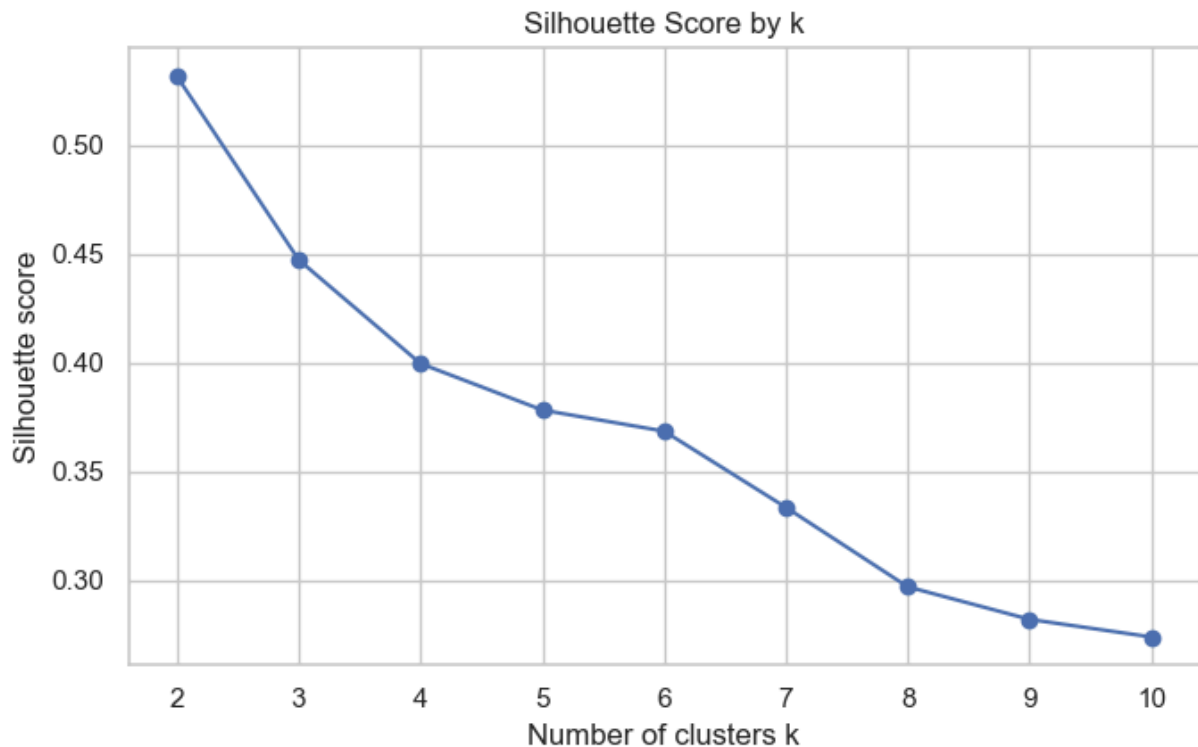
for k in range(2, 11):
    runs = [kmeans(X, k=k, seed=seed) for seed in range(10)]
    best_centroids, best_labels, best_inertia = min(runs, key=lambda run: run[2])
    score = silhouette_score(X, best_labels)
    best_runs_by_k[k] = (best_centroids, best_labels, best_inertia)
    silhouette_rows.append({"k": k, "silhouette": score, "best_inertia": best_inertia})

silhouette_df = pd.DataFrame(silhouette_rows)
display(silhouette_df.round(3))

plt.figure(figsize=(7, 4.5))
plt.plot(silhouette_df["k"], silhouette_df["silhouette"], marker="o")
plt.xticks(range(2, 11))
plt.title("Silhouette Score by k")
plt.xlabel("Number of clusters k")
plt.ylabel("Silhouette score")
plt.tight_layout()
plt.show()

best_silhouette_row = silhouette_df.loc[silhouette_df["silhouette"].idxmax()]
print(f"Best silhouette k: {int(best_silhouette_row['k'])} with score {best_silhouette_row['silhouette']}")
```

	k	silhouette	best_inertia
0	2	0.532	565.708
1	3	0.447	379.393
2	4	0.400	300.400
3	5	0.378	232.597
4	6	0.369	205.127
5	7	0.334	187.155
6	8	0.297	171.368
7	9	0.282	160.739
8	10	0.274	152.667



Best silhouette k: 2 with score 0.532

Silhouette is maximized at `k=2`, which agrees with the strongest elbow bend but not with the three recorded species. I would trust silhouette more for the strictly unsupervised question of how many compact, well-separated morphology groups are present; I would still inspect `k=3` because the biological labels tell us there are three species and the third cluster captures meaningful Adelie/Chinstrap structure.

(g) Sensitivity to Initialization

```
In [8]: seed_rows = []
for seed in range(50):
    centroids_seed, labels_seed, inertia_seed = kmeans(X, k=3, seed=seed)
    seed_rows.append(
        {
            "seed": seed,
            "inertia": inertia_seed,
            "centroids": centroids_seed,
            "labels": labels_seed,
        }
    )

seed_df = pd.DataFrame(seed_rows)
best_seed_row = seed_df.loc[seed_df["inertia"].idxmin()]
worst_seed_row = seed_df.loc[seed_df["inertia"].idxmax()]
inertia_spread = seed_df["inertia"].max() - seed_df["inertia"].min()

print(f"Lowest inertia: {best_seed_row['inertia']:.3f} at seed {int(best_seed_row['seed'])}")
print(f"Highest inertia: {worst_seed_row['inertia']:.3f} at seed {int(worst_seed_row['seed'])}")
print(f"Spread: {inertia_spread:.3f}")
```

```

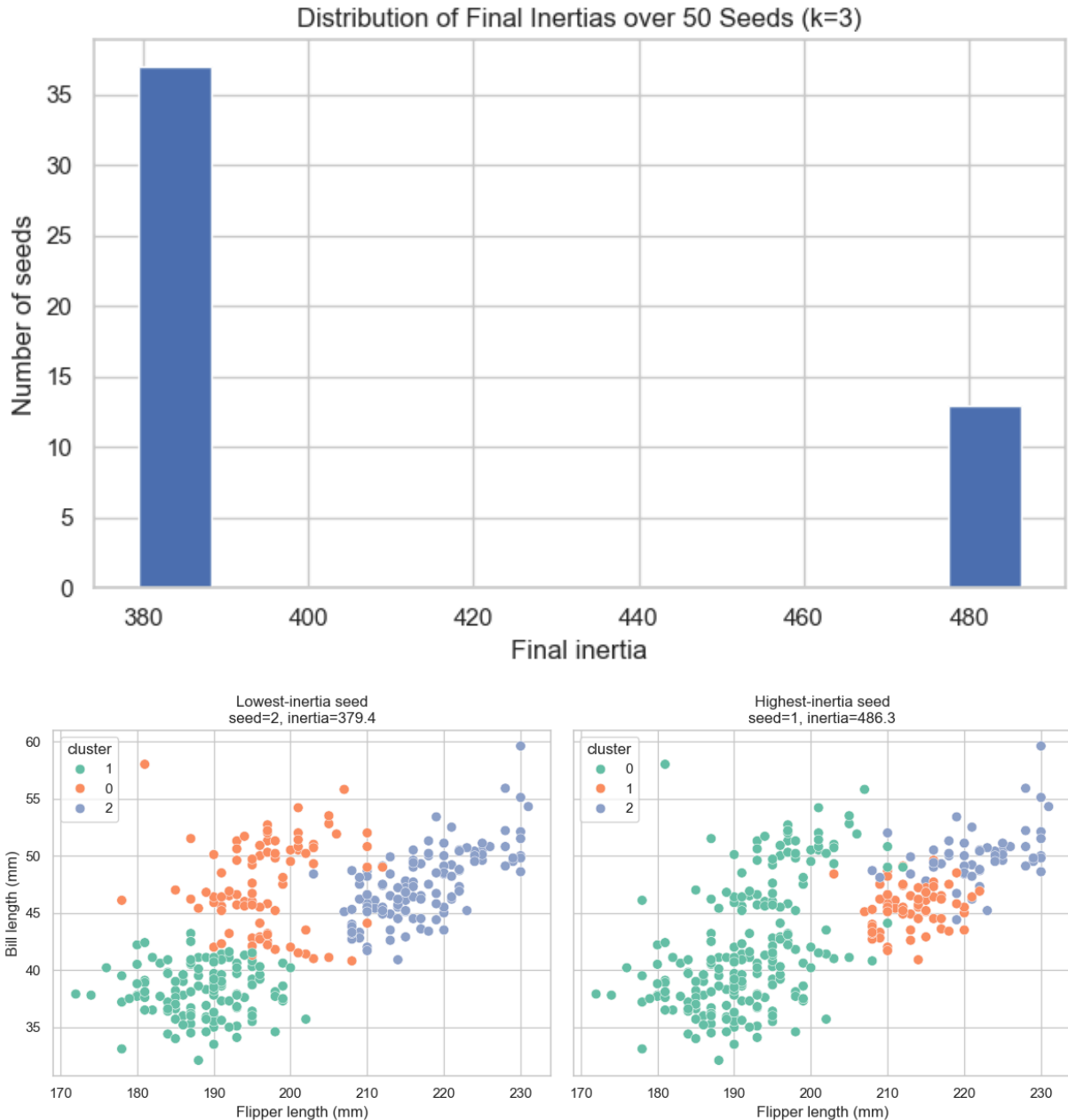
plt.figure(figsize=(7, 4.5))
plt.hist(seed_df["inertia"], bins=12, edgecolor="white")
plt.title("Distribution of Final Inertias over 50 Seeds (k=3)")
plt.xlabel("Final inertia")
plt.ylabel("Number of seeds")
plt.tight_layout()
plt.show()

fig, axes = plt.subplots(1, 2, figsize=(12, 5), sharex=True, sharey=True)
for ax, row, title in [
    (axes[0], best_seed_row, "Lowest-inertia seed"),
    (axes[1], worst_seed_row, "Highest-inertia seed"),
]:
    temp_df = clean_penguins.assign(cluster=row["labels"].astype(str))
    sns.scatterplot(
        data=temp_df,
        x="flipper_length_mm",
        y="bill_length_mm",
        hue="cluster",
        palette="Set2",
        s=60,
        edgecolor="white",
        linewidth=0.4,
        ax=ax,
        legend=True,
    )
    ax.set_title(f"{title}\nseed={int(row['seed'])}, inertia={row['inertia']:.1f}")
    ax.set_xlabel("Flipper length (mm)")
    ax.set_ylabel("Bill length (mm)")

plt.tight_layout()
plt.show()

```

Lowest inertia: 379.393 at seed 2
 Highest inertia: 486.297 at seed 1
 Spread: 106.905



The spread is large enough to matter: some seeds find a strong three-species-like solution, while others split the Gentoo-like birds into two clusters and merge much of Adelie/Chinstrap. The two side-by-side clusterings therefore disagree substantially, not just on a few boundary points.

(h) Summary and Recommendation

The morphology data show a very strong split into two compact groups, with Gentoo separated from the other penguins, and a plausible but less clean three-cluster structure corresponding to the three recorded species. The elbow curve has its strongest bend at $k=2$ and a secondary bend around $k=3$, while silhouette selects $k=2$, so I am most confident in two broad morphology groups and moderately confident in a species-level $k=3$ clustering. Using $k=3$, the best fixed run in this notebook recovered the recorded

species with purity around 0.91, with most errors occurring between Adelie and Chinstrap. Initialization matters: across 50 random seeds, Lloyd's algorithm landed in noticeably different local minima, so in practice I would run many initializations and keep the lowest-inertia solution. A limitation is that this analysis uses only four morphology measurements and drops rows with missing morphology values rather than modeling or imputing missingness.